# Rubrica: Systems Design

## Learning a black box, one proven rule at a time

Reto 2 · La caja negra · rubrica.dev · Kevin Vicent · v15

The black box is an unknown decision procedure. We find its rules with experiments. We prove each rule against the real service. Then we compile the rules into a deterministic replica. Most agent systems need an LLM judge. Ours has an exact judge for every behavior it probes: the original service. What it never probes is the open risk, so we measure it.

## 1. Problem

An undocumented service receives text messages on `POST /msg` (body `{"m": "…"}`) and keeps state between requests. It also has `/__reset` and `/health`. Each token has its own state.

We must build a replacement. After `/__reset`, each response must have **the same status and body** as the original, errors and unknown routes included. The tests do not compare headers or JSON key order. There are **100 hidden tests**: 40 single requests and 60 sequences of 2–12 steps. Every observable behavior is part of the contract (Hyrum's Law). The replica must not call the original. **The service changes during the day, without notice. The tests use the 20:00 version.**

**Design principle:** the README contract is the architecture. Everything the service returns is **data**: learned, versioned and replaceable. No part of the design depends on a specific operation, field or error code.

| Assumption | How we test it |
|---|---|
| Same state and same request give the same response | Replay a sequence twice |
| Messages have learnable structure (operations, fields, checks) | Fit rules; confirm with probes |
| When several errors apply, one fixed precedence decides | Combine errors in pairs |
| Generated values (IDs, counters) follow a reproducible rule | Compare sequences across resets |
| Tests send one request at a time | README: reset before each test |

**Hard cases:** values the service computes (for example integrity checks) · error precedence · generated IDs · repeated operations · numeric limits and boundaries · auth (team token, other token, no token) · unknown routes and methods · **behavior changes during the day**.

**Out of scope:** headers (the tests do not compare them) · latency under the 5 s step limit.

## 2. Architecture: dual-process, one ontology

We use Kahneman's terms. **System 2** is slow and deliberate: the offline loop that discovers the rules. **System 1** is fast and automatic: the deployed replica that applies them.

1. **Agent harness (System 2, offline).** A person and a reasoning model use only recorded tools (`probe`, `sequence`, `fuzz`, `shrink`, `diff`); every request is logged with its hypothesis and `X-Req`. **Rule: no request without a prediction:** a code gate blocks any request that has no prediction, that the log already answers, or that exceeds the budget.
2. **Hypothesis search, not answer search.** Blind sweeps waste budget (our first one cost 543 requests). The harness fits candidate rules offline and sends the request that eliminates the most. **Target: fewer than 50 probes per unknown rule;** if no candidate survives, the rule becomes *supuesto* (a documented default).
3. **Service ontology (memory).** An *ontology* is an explicit model of a domain: its concepts and the relations between them. Ours models the black box: **operations, fields, state, rules, errors and evidence**. A rule applies to an operation, needs preconditions, returns a response, changes state, outranks other rules and is proven by log lines. Confidence uses the README's levels: *confirmado* (proven, repeated), *probable* (seen once or inferred), *supuesto* (untested default). **We use it to** run the replica, generate `SPEC.md`, build tests from evidence, choose the next probes from its gaps, and **compare versions: most changes are new data; a new operation is one more rule.**
4. **Replica (System 1, runtime).** HTTP adapter → serial queue → decision core. **The first matching rule decides; the precedence order is learned and versioned in the ontology.** State is in memory, one state per token.

| Decision | Chosen | Rejected, and why |
|---|---|---|
| Runtime | **Compiled rule table:** System 2 knowledge made automatic; no probabilities at runtime | *A probabilistic layer (LLM or classifier):* a formal protocol has nothing to perceive; not exact or repeatable. *Record and replay:* fails on new inputs. |
| Discovery | **A person and a model in a harness**, exact oracle | *Free LLM chat:* no proof, no log. *Autonomous agent:* false spec claims lose points. *Manual curl:* too slow. |
| Knowledge | **Typed ontology**, one source for code, spec and tests | *Separate spec, code and tests:* they diverge after the change. *RAG over logs:* approximate recall. *OWL/RDF tooling:* heavy for one day; typed records hold the same relations. |
| Checks | **Code for every exact check, cheapest first:** cache → corpus → live original. A **System One classifier** only if code cannot tell drift from noise. | *LLM judge as verifier:* false passes. The original is a free, exact judge. |
| Server and hosting | **Minimal server we control** (exact bodies, also for unknown routes); **one always-on instance** | *Framework defaults:* different 404/405 bodies. *Autoscaling:* each instance has its own state. |

## 3. How we measure: evals first

The **recorded corpus** guides fixes; **held-out tests** only measure, so the score stays honest (Goodhart's Law). A different generator builds them, so they do not share our blind spots.

- **Differential test:** replays every recorded sequence against the replica and compares status and body with the original's answers. Target: 100% on the corpus and the README examples; **≥ 95% on held-out tests**.
- **Drift canary:** every 10 min, sends 10 requests to the original: 5 fixed on the most-tested paths and 5 that rotate through the recorded corpus. A difference with the recorded answer means the service changed. Detect in ≤ 10 min on the fixed paths and within one rotation elsewhere; fix in ≤ 45 min.
- **Coverage:** operation × precondition × error matrix; low-confidence rules and empty cells first. **Discovery rate:** new behaviors per 100 probes; an area is done when it reaches zero. **Budget (README): 8,000 requests/day, 10/s per token**, spent in this order: discovery, fuzz, canary, reserve.
- **Seeded tests:** each error alone and **in pairs**, boundary values ±1, case and whitespace variants, field count ±1, repeated operations, limits.
- **3 validations:** only if the exact-match rate did not decrease (~15:15, ~17:15, ~19:30).

**Risks:** a rule outside our candidate family (add candidates, probe again) · budget runs out (cache, never repeat) · late drift (canary runs until 20:00).